DevFlow OS

Complete Build Guide — From Zero to Full Stack

React · TypeScript · Express · REST API

How to Use This Guide

This document is your single source of truth. You should not need anything else to build this project from scratch. Follow it in order — do not skip phases, do not read ahead before you've completed the current phase.

Rule #1 — Build first, read later
Every concept in this guide is introduced exactly when you need it. If you haven't finished Phase 2, don't read Phase 3.

Rule #2 — Stuck for 30 minutes?
Simplify. Remove code until something works, then add back piece by piece. Never add complexity to solve confusion.

Rule #3 — No passive learning
Every docs link in this guide = read it, then immediately write code using it. Reading without coding is wasted time.

Tech Stack — Final Decisions

These decisions are made for you. Do not debate them.

Technology	Choice
Frontend framework	React 18 with TypeScript
Build tool	Vite (fast, simple config)
Styling	Tailwind CSS
Form management	React Hook Form
Validation	Zod
Nested state updates	Immer (Phase 3 only, when needed)

Backend	Node.js + Express
Database	In-memory (JSON array) — no DB needed
IDs	uuid package
HTTP client	fetch() — no axios needed

Why Express, not Firebase?
Firebase hides the HTTP layer — the exact thing you need to learn. Express forces you to understand requests, responses, routes, and error handling. In 3 weeks you will be able to build a backend for anything.

What to Learn and When

Do not learn everything upfront. This table tells you exactly when to open each doc and how long to spend.

Topic	When	Time budget	Link
React + TypeScript basics	Before Phase 1	2 hrs max	react.dev/learn
useState hook	Before Phase 3	30 min	react.dev/reference/usestate
useEffect hook	Before Phase 5	45 min	react.dev/reference/useeffect
TypeScript basics	Before Phase 1	1 hr	typescriptlang.org/docs
Tailwind CSS	Start of Phase 2	30 min	tailwindcss.com/docs
React Hook Form	Start of Phase 3 forms	45 min	react-hook-form.com/get-started
Zod	Same day as RHF	30 min	zod.dev
Express.js basics	Before Phase 4	1 hr	expressjs.com/starter
Express routing	Before Phase 4	30 min	expressjs.com/routing
fetch() + async/await	Before Phase 5	30 min	mdn fetch API
AbortController	Phase 5 — hooks	20 min	mdn AbortController
Immer	Phase 3 — subtasks only	20 min	immerjs.github.io
TypeScript generics	Phase 5 — API client	30 min	TS handbook generics

---

Folder Structure — Set This Up in Phase 1

Create this exact structure on day one. Never rename or reorganize it mid-project.

```
devflow-os/
├── /client                    ← React frontend
│   ├── /src
│   │   ├── /features          ← one folder per module
│   │   │   ├── /tasks
│   │   │   │   ├── TaskList.tsx
│   │   │   │   ├── TaskItem.tsx
│   │   │   │   ├── TaskForm.tsx
│   │   │   │   ├── tasks.schema.ts    ← Zod schema
│   │   │   │   └── useTasks.ts        ← custom hook (Phase 5)
│   │   │   ├── /notes
│   │   │   │   ├── NoteList.tsx
│   │   │   │   ├── NoteCard.tsx
│   │   │   │   ├── NoteForm.tsx
│   │   │   │   ├── notes.schema.ts
│   │   │   │   └── useNotes.ts
│   │   │   └── /expenses
│   │   │       ├── ExpenseList.tsx
│   │   │       ├── ExpenseRow.tsx
│   │   │       ├── ExpenseForm.tsx
│   │   │       ├── expenses.schema.ts
│   │   │       └── useExpenses.ts
│   │   ├── /services
│   │   │   ├── api.ts             ← generic HTTP client
│   │   │   ├── tasks.api.ts
│   │   │   ├── notes.api.ts
│   │   │   └── expenses.api.ts
│   │   ├── /hooks
│   │   │   └── useDataFetch.ts    ← reusable fetch hook
│   │   ├── /types
│   │   │   └── index.ts           ← Task, Note, Expense
│   │   ├── /components
│   │   │   ├── Layout.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   └── ThemeToggle.tsx
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   └── package.json
│
└── /server                    ← Express backend
    ├── /routes
    │   ├── tasks.js
    │   ├── notes.js
    │   └── expenses.js
    ├── server.js              ← entry point
    └── package.json
```

TypeScript Types — Define These First

File: client/src/types/index.ts — write this before any component.

```
// client/src/types/index.ts

export interface Task {
```

```
  id: string;
  title: string;
  completed: boolean;
  priority: 'low' | 'medium' | 'high';
  deadline?: string;        // ISO date string e.g. '2025-04-15'
  subtasks: Task[];         // recursive — same shape
}

export interface Note {
  id: string;
  title: string;
  content: string;
  createdAt: string;
}

export interface Expense {
  id: string;
  amount: number;
  category: string;
  description: string;
  date: string;
}
```

Phase 1: Project Setup [1–2 hours]

What you're building

A blank Vite + React + TypeScript app with Tailwind configured and the full folder structure created. No components, no logic — just a working dev environment.

Step-by-step commands

1. Scaffold the frontend

```
npm create vite@latest client -- --template react-ts
cd client
npm install
npm run dev   # should open localhost:5173 with Vite default page
```

2. Install all frontend dependencies upfront

```
npm install react-hook-form zod @hookform/resolvers
npm install immer
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

3. Configure Tailwind

Edit tailwind.config.js:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  darkMode: 'class',
  theme: { extend: {} },
  plugins: [],
};
```

Replace src/index.css with just three lines:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

4. Create the folder structure

```
mkdir -p src/features/tasks src/features/notes src/features/expenses
mkdir -p src/services src/hooks src/types src/components
```

5. Verify TypeScript is strict

Open tsconfig.json and confirm these are set:

```
"strict": true,
"noUnusedLocals": true,
"noUnusedParameters": true,
```

Phase 1 done when:

npm run dev runs with no errors. Tailwind class works (add className="text-blue-500" to a div and see blue text). Folder structure exists.

Failure signal:

If you spent more than 2 hours here — stop. Config issues are a trap. Delete and re-scaffold. The commands above are all you need.

Phase 2: Static UI — All Three Modules [2–4 hours]

What you're building

All three modules rendered with hardcoded fake data. No state, no events, no API calls. Just layout and Tailwind styling. The goal is to see the full UI before writing a single line of logic.

> **Why start with static UI?**
> Hardcoded UI forces you to design your component structure before adding complexity. If the layout doesn't work with fake data, it won't work with real data either.

Component breakdown

Component	Responsibility
Layout.tsx	Sidebar + main content area
Sidebar.tsx	Navigation links: Tasks, Notes, Expenses
TaskList.tsx	Renders array of TaskItem components
TaskItem.tsx	Single task: title, priority badge, deadline, checkbox
NoteList.tsx	Grid/list of NoteCard components
NoteCard.tsx	Single note: title, content preview, date
ExpenseList.tsx	List of ExpenseRow components + total at top
ExpenseRow.tsx	Single expense: description, amount, category, date

App.tsx — hardcoded to start

```
// src/App.tsx — Phase 2 version, no state yet

import { Layout } from './components/Layout';
import { TaskList } from './features/tasks/TaskList';
import type { Task } from './types';

const FAKE_TASKS: Task[] = [
  { id: '1', title: 'Set up Express server', completed: false,
    priority: 'high', deadline: '2025-04-15', subtasks: [] },
  { id: '2', title: 'Build TaskForm component', completed: true,
    priority: 'medium', deadline: undefined, subtasks: [] },
];

function App() {
  return (
    <Layout>
      <TaskList tasks={FAKE_TASKS} />
```

```
    </Layout>
  );
}

export default App;
```

TaskItem.tsx — example component

```
// src/features/tasks/TaskItem.tsx

import type { Task } from '../../types';

interface Props {
  task: Task;
}

const priorityColors = {
  low:    'bg-green-100 text-green-800',
  medium: 'bg-yellow-100 text-yellow-800',
  high:   'bg-red-100 text-red-800',
};

export function TaskItem({ task }: Props) {
  return (
    <div className="flex items-start gap-3 p-4 border-b border-gray-100">
      <input type="checkbox" checked={task.completed} readOnly
             className="mt-1 w-4 h-4" />
      <div className="flex-1">
        <p className={`font-medium ${task.completed ? 'line-through text-gray-
400' : ''}`}>
          {task.title}
        </p>
        <div className="flex gap-2 mt-1">
          <span className={`text-xs px-2 py-0.5 rounded-full $
{priorityColors[task.priority]}`}>
            {task.priority}
          </span>
          {task.deadline && (
            <span className="text-xs text-gray-400">{task.deadline}</span>
          )}
        </div>
      </div>
    </div>
  );
}
```

Component size rules

Signal	Action
Too big — split it	Component renders 3+ different sections, or file is over 100 lines, or you're passing 5+ props
Right size	Does one thing, has one reason to change, fits in ~60 lines
Too small — keep inline	Pure wrapper with no logic (<5 lines) — inline it

Phase 2 done when:

All three views render with hardcoded data. Navigate between Tasks, Notes, Expenses in the sidebar. No JS errors in console. Layout works on mobile width (375px).

Phase 3: Local State + Full CRUD [3–5 hours]

What you're building

Full create/edit/delete for all three modules. All data lives in React state (memory only — resets on refresh). Forms with validation. Theme toggle persisted to localStorage. No backend yet.

State management rules — memorise these

Rule: Never store derived values

If you can calculate it from existing state, do not put it in useState.

```
// WRONG — storing a value you can calculate
const [tasks, setTasks] = useState<Task[]>([]);
const [taskCount, setTaskCount] = useState(0);   // ← never do this

// RIGHT — derive it
const [tasks, setTasks] = useState<Task[]>([]);
const completedCount = tasks.filter(t => t.completed).length;  // ← derived
```

Rule: One useState per data type

tasks, notes, expenses each get their own useState. Do not merge them into one object.

Zod schemas — write these before your forms

tasks.schema.ts

```
// src/features/tasks/tasks.schema.ts

import { z } from 'zod';

export const taskSchema = z.object({
  title:    z.string().min(1, 'Title is required').max(100, 'Too long'),
  priority: z.enum(['low', 'medium', 'high']),
  deadline: z.string().optional(),
});

export type TaskFormData = z.infer<typeof taskSchema>;
// TaskFormData is now: { title: string; priority: 'low'|'medium'|'high';
deadline?: string }
```

notes.schema.ts

```
import { z } from 'zod';
```

```
export const noteSchema = z.object({
  title:   z.string().min(1, 'Title is required').max(80),
  content: z.string().min(1, 'Content cannot be empty'),
});

export type NoteFormData = z.infer<typeof noteSchema>;
```

expenses.schema.ts

```
import { z } from 'zod';

export const expenseSchema = z.object({
  description: z.string().min(1, 'Description is required'),
  amount:      z.number({ invalid_type_error: 'Must be a
number' }).positive('Must be positive'),
  category:    z.string().min(1, 'Pick a category'),
  date:        z.string().min(1, 'Date is required'),
});

export type ExpenseFormData = z.infer<typeof expenseSchema>;
```

TaskForm.tsx — the full form pattern

Once you write this correctly for tasks, notes and expenses follow the exact same pattern.

```
// src/features/tasks/TaskForm.tsx

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { taskSchema, type TaskFormData } from './tasks.schema';

interface Props {
  onSubmit: (data: TaskFormData) => void;
  onCancel: () => void;
}

export function TaskForm({ onSubmit, onCancel }: Props) {
  const {
    register,
    handleSubmit,
    formState: { errors },
    reset,
  } = useForm<TaskFormData>({
    resolver: zodResolver(taskSchema),
    defaultValues: { priority: 'medium' },
  });

  const handleValidSubmit = (data: TaskFormData) => {
    onSubmit(data);
    reset();
  };

  return (
    <form onSubmit={handleSubmit(handleValidSubmit)} className="space-y-4 p-4
border rounded">
      <div>
```

```
        <input
          {...register('title')}
          placeholder="Task title"
          className="w-full px-3 py-2 border rounded"
        />
        {errors.title && (
          <p className="text-red-500 text-sm mt-1">{errors.title.message}</p>
        )}
      </div>

      <select {...register('priority')} className="w-full px-3 py-2 border
rounded">
        <option value="low">Low</option>
        <option value="medium">Medium</option>
        <option value="high">High</option>
      </select>

      <input {...register('deadline')} type="date" className="w-full px-3 py-2
border rounded" />

      <div className="flex gap-2">
        <button type="submit" className="px-4 py-2 bg-blue-600 text-white
rounded">
          Add Task
        </button>
        <button type="button" onClick={onCancel} className="px-4 py-2 border
rounded">
          Cancel
        </button>
      </div>
    </form>
  );
}
```

State in App.tsx — Phase 3

```
// src/App.tsx — Phase 3 version

import { useState } from 'react';
import { v4 as uuid } from 'uuid';
import type { Task, Note, Expense } from './types';
import type { TaskFormData } from './features/tasks/tasks.schema';

function App() {
  const [tasks, setTasks]       = useState<Task[]>([]);
  const [notes, setNotes]       = useState<Note[]>([]);
  const [expenses, setExpenses] = useState<Expense[]>([]);
  const [theme, setTheme]       = useState<'light' | 'dark'>(
    () => (localStorage.getItem('theme') as 'light' | 'dark') ?? 'light'
  );

  // Derived — never stored
  const completedCount = tasks.filter(t => t.completed).length;

  function addTask(data: TaskFormData) {
    const newTask: Task = { ...data, id: uuid(), completed: false, subtasks: []
};
    setTasks(prev => [...prev, newTask]);
  }
```

```
  function toggleTask(id: string) {
    setTasks(prev =>
      prev.map(t => t.id === id ? { ...t, completed: !t.completed } : t)
    );
  }

  function deleteTask(id: string) {
    setTasks(prev => prev.filter(t => t.id !== id));
  }

  function toggleTheme() {
    setTheme(prev => {
      const next = prev === 'light' ? 'dark' : 'light';
      localStorage.setItem('theme', next);
      return next;
    });
  }

  // ... pass handlers down to components as props
}
```

When to use Immer — subtasks only

Do NOT install Immer until you've written this update logic manually and felt the pain:

```
// Without Immer — toggle a subtask (painful)
setTasks(prev => prev.map(task =>
  task.id === taskId
    ? {
        ...task,
        subtasks: task.subtasks.map(sub =>
          sub.id === subId ? { ...sub, completed: !sub.completed } : sub
        )
      }
    : task
));
```

After you've written the above and suffered — install Immer:

```
npm install immer   # (already done in Phase 1)
```

```
// With Immer — same logic, readable
import { produce } from 'immer';

setTasks(produce(draft => {
  const task = draft.find(t => t.id === taskId);
  const sub  = task?.subtasks.find(s => s.id === subId);
  if (sub) sub.completed = !sub.completed;
}));
```

Phase 3 done when:

Create a task. Toggle it complete. Delete it. Add a subtask. Toggle the subtask. All forms show validation errors when submitted empty. Theme toggle works and persists after refresh.

Phase 4: Express Backend [3–5 hours]

What you're building

A Node.js + Express REST API with full CRUD for tasks, notes, and expenses. Data lives in memory (resets on server restart — that's fine for now). Test every route before touching the frontend.

Setup

```
# From the project root
mkdir server && cd server
npm init -y
npm install express cors uuid
```

server/server.js — entry point

```
const express = require('express');
const cors    = require('cors');

const taskRoutes    = require('./routes/tasks');
const noteRoutes    = require('./routes/notes');
const expenseRoutes = require('./routes/expenses');

const app = express();

app.use(cors());           // allow requests from localhost:5173
app.use(express.json());   // parse JSON request bodies

app.use('/api/tasks',    taskRoutes);
app.use('/api/notes',    noteRoutes);
app.use('/api/expenses', expenseRoutes);

// Central error handler — every route can call next(err)
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).json({ error: err.message });
});

const PORT = 3001;
app.listen(PORT, () => console.log(`DevFlow API on :${PORT}`));
```

server/routes/tasks.js — write this, then copy for notes + expenses

```
const express = require('express');
const { v4: uuid } = require('uuid');

const router = express.Router();
```

```
let tasks = [];  // in-memory store

// GET /api/tasks — return all
router.get('/', (req, res) => {
  res.json(tasks);
});

// POST /api/tasks — create one
router.post('/', (req, res) => {
  const task = {
    id:        uuid(),
    completed: false,
    subtasks:  [],
    ...req.body,   // spread client's fields over defaults
  };
  tasks.push(task);
  res.status(201).json(task);
});

// PUT /api/tasks/:id — update one
router.put('/:id', (req, res) => {
  const index = tasks.findIndex(t => t.id === req.params.id);
  if (index === -1) return res.status(404).json({ error: 'Task not found' });
  tasks[index] = { ...tasks[index], ...req.body };
  res.json(tasks[index]);
});

// DELETE /api/tasks/:id — remove one
router.delete('/:id', (req, res) => {
  const index = tasks.findIndex(t => t.id === req.params.id);
  if (index === -1) return res.status(404).json({ error: 'Task not found' });
  tasks.splice(index, 1);
  res.status(204).send();
});

module.exports = router;
```

Test your routes — do this before touching the frontend

Run the server: node server.js

Then test each route:

```
# Get all tasks (should return [])
curl http://localhost:3001/api/tasks

# Create a task
curl -X POST http://localhost:3001/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Test task","priority":"high"}'

# Get all (should show your new task)
curl http://localhost:3001/api/tasks

# Update it (replace ID with one from GET response)
curl -X PUT http://localhost:3001/api/tasks/YOUR_ID \
  -H "Content-Type: application/json" \
  -d '{"completed":true}'
```

```
# Delete it
curl -X DELETE http://localhost:3001/api/tasks/YOUR_ID

# Confirm deleted
curl http://localhost:3001/api/tasks
```

Phase 4 done when:

All 4 routes for all 3 modules (12 total) return correct responses when tested with curl or Postman. Server starts with no errors. Do NOT move to Phase 5 until this is confirmed.

Common mistakes here:

Forgetting app.use(express.json()) — req.body will be undefined. Forgetting cors() — frontend will get blocked. Testing routes in the browser instead of curl — GET works but POST/PUT/DELETE won't.

Phase 5: Frontend ↔ Backend Integration [3–5 hours]

What you're building

The generic API client. The reusable useDataFetch hook. Wire all three modules to the backend. Add loading and error states to the UI. Handle request cancellation with AbortController.

The generic API client

File: client/src/services/api.ts

```
// src/services/api.ts
// Generic HTTP client — handles all fetch() logic in one place

const BASE_URL = 'http://localhost:3001/api';

// T = the type of data you expect back (e.g. Task, Note[], Expense)
async function request<T>(
  endpoint: string,
  options?: RequestInit,
  signal?: AbortSignal
): Promise<T> {
  const response = await fetch(`${BASE_URL}${endpoint}`, {
    headers: { 'Content-Type': 'application/json' },
    signal,
    ...options,
  });

  if (!response.ok) {
    throw new Error(`${response.status}: ${response.statusText}`);
  }

  // 204 No Content (DELETE) has no body
  if (response.status === 204) return undefined as T;

  return response.json();
}

export const api = {
  get:    <T>(url: string, signal?: AbortSignal) =>
            request<T>(url, { method: 'GET' }, signal),

  post:   <T>(url: string, body: unknown) =>
            request<T>(url, { method: 'POST', body: JSON.stringify(body) }),

  put:    <T>(url: string, body: unknown) =>
            request<T>(url, { method: 'PUT', body: JSON.stringify(body) }),

  delete: <T>(url: string) =>
            request<T>(url, { method: 'DELETE' }),
};
```

The reusable fetch hook

File: client/src/hooks/useDataFetch.ts

```
// src/hooks/useDataFetch.ts

import { useState, useEffect } from 'react';
import { api } from '../services/api';

interface FetchState<T> {
  data:    T | null;
  loading: boolean;
  error:   string | null;
  refetch: () => void;
}

export function useDataFetch<T>(url: string): FetchState<T> {
  const [data,    setData]    = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState<string | null>(null);
  const [tick,    setTick]    = useState(0);  // increment to refetch

  useEffect(() => {
    const controller = new AbortController();

    (async () => {
      setLoading(true);
      setError(null);
      try {
        const result = await api.get<T>(url, controller.signal);
        setData(result);
      } catch (err) {
        // Don't set error if the request was intentionally cancelled
        if ((err as Error).name !== 'AbortError') {
          setError((err as Error).message);
        }
      } finally {
        setLoading(false);
      }
    })();

    // Cleanup: cancel the in-flight request on unmount or re-run
    return () => controller.abort();
  }, [url, tick]);

  const refetch = () => setTick(t => t + 1);

  return { data, loading, error, refetch };
}
```

Per-module API service

File: client/src/services/tasks.api.ts — write this, then copy for notes and expenses.

```
// src/services/tasks.api.ts

import { api } from './api';
import type { Task } from '../types';
import type { TaskFormData } from '../features/tasks/tasks.schema';
```

```
export const tasksApi = {
  getAll:  ()                  => api.get<Task[]>('/tasks'),
  create:  (data: TaskFormData) => api.post<Task>('/tasks', data),
  update:  (id: string, data: Partial<Task>) => api.put<Task>(`/tasks/${id}`,
data),
  remove:  (id: string)        => api.delete<void>(`/tasks/${id}`),
};
```

Using the hook in a feature component

```
// src/features/tasks/TaskList.tsx — Phase 5 version

import { useDataFetch } from '../../hooks/useDataFetch';
import { tasksApi } from '../../services/tasks.api';
import type { Task } from '../../types';
import type { TaskFormData } from './tasks.schema';
import { TaskItem } from './TaskItem';
import { TaskForm } from './TaskForm';

export function TaskList() {
  const { data: tasks, loading, error, refetch } =
useDataFetch<Task[]>('/tasks');

  async function handleCreate(formData: TaskFormData) {
    await tasksApi.create(formData);
    refetch();  // re-fetch list after mutation
  }

  async function handleToggle(task: Task) {
    await tasksApi.update(task.id, { completed: !task.completed });
    refetch();
  }

  async function handleDelete(id: string) {
    await tasksApi.remove(id);
    refetch();
  }

  if (loading) return <div className="p-4 text-gray-500">Loading
tasks...</div>;
  if (error)   return <div className="p-4 text-red-500">Error: {error}</div>;

  return (
    <div>
      <TaskForm onSubmit={handleCreate} onCancel={() => {}} />
      {tasks?.map(task => (
        <TaskItem
          key={task.id}
          task={task}
          onToggle={handleToggle}
          onDelete={handleDelete}
        />
      ))}
    </div>
  );
}
```

useEffect rules — when to use it and when not to

Situation	Use useEffect?
Fetch data on mount	YES — useEffect with [] + AbortController cleanup
Sync theme to localStorage	YES — useEffect with [theme] dependency
Set state based on other state	NO — derive it or restructure state
Count completed tasks	NO — derive: tasks.filter(t=>t.completed).length
Timer / setInterval	YES — useEffect with clearInterval cleanup
Respond to a button click	NO — use an event handler, not useEffect

Phase 5 done when:
Open the app. Data loads from backend. Create a task. Refresh the page. Task is still there. Kill the server. The app shows an error message (not a blank screen). Restart server. Data loads again.

Common Mistakes — Read This When Stuck

1. Storing derived state

```
// WRONG — taskCount duplicates info already in tasks array
const [taskCount, setTaskCount] = useState(0);

// RIGHT — derive it inline, no state needed
const taskCount = tasks.length;
const completedCount = tasks.filter(t => t.completed).length;
```

2. Missing useEffect cleanup

```
// WRONG — if component unmounts while fetch is in flight, React
// will try to set state on an unmounted component
useEffect(() => {
  fetch('/api/tasks').then(r => r.json()).then(setTasks);
}, []);

// RIGHT — cancel the request on cleanup
useEffect(() => {
  const controller = new AbortController();
  fetch('/api/tasks', { signal: controller.signal })
    .then(r => r.json())
    .then(setTasks)
    .catch(err => {
      if (err.name !== 'AbortError') setError(err.message);
    });
  return () => controller.abort();  // ← this line is required
}, []);
```

3. Components importing from services directly

```
// WRONG — TaskItem shouldn't know about the API
// src/features/tasks/TaskItem.tsx
import { tasksApi } from '../../services/tasks.api';  // ← wrong

// RIGHT — TaskItem receives handlers as props from its parent
interface Props {
  task: Task;
  onToggle: (task: Task) => void;  // parent handles the API call
  onDelete: (id: string) => void;
}
```

4. Using useEffect to sync state with state

```
// WRONG — setFilteredTasks every time tasks changes
const [filteredTasks, setFilteredTasks] = useState<Task[]>([]);
useEffect(() => {
  setFilteredTasks(tasks.filter(t => !t.completed));
}, [tasks]);
```

```
// RIGHT — derive it inline, no extra state or effect
const filteredTasks = tasks.filter(t => !t.completed);
```

5. Race conditions on fast re-renders

Covered by using AbortController in useDataFetch — the cleanup function cancels any in-flight request when the component re-renders or unmounts. This is why the cleanup return is mandatory.

Milestone Checklist

Phase 1 — Setup

- npm run dev works with no errors
- Adding className="text-blue-500" shows blue text — Tailwind works
- All folders exist in src/
- TypeScript strict mode is on

Phase 2 — Static UI

- Navigate between Tasks, Notes, Expenses in sidebar
- Hardcoded tasks render with priority badges and deadlines
- Hardcoded notes render with title and content preview
- Hardcoded expenses render with amounts and categories
- No JavaScript errors in browser console

Phase 3 — Local State

- Create a task → appears in list
- Toggle task complete → line-through style applies
- Delete task → disappears from list
- Submit form with empty title → shows validation error
- Add subtask to a task → subtask appears nested
- Toggle theme → changes to dark mode
- Refresh page → theme preference preserved, tasks gone (expected)

Phase 4 — Backend

- node server.js starts with no errors

- curl GET /api/tasks returns []
- curl POST /api/tasks creates a task with an auto-generated id
- curl GET /api/tasks returns the created task
- curl PUT /api/tasks/:id updates the task
- curl DELETE /api/tasks/:id removes it
- Same for /api/notes and /api/expenses

Phase 5 — Integration

- App loads and shows tasks from the backend
- Create a task in the UI → visible in the list
- Refresh the browser → task is still there (backend persisted it)
- Delete a task → gone after page refresh
- Kill the backend server → app shows error message, not a crash
- Restart backend → app recovers (either on next action or auto-refetch)
- Network tab in DevTools shows actual HTTP requests

Final Tips

When stuck
Go back to the last thing that worked. Remove everything you added since then. Add it back one line at a time.

When TypeScript errors feel endless
Fix them in order. The first error often causes five others. Fix top to bottom, not the easiest-looking ones first.

When the API doesn't connect
Check: is the server running? Is the port 3001? Is cors() middleware added? Open the Network tab in DevTools and read the actual error.

When to add a database
Never — for this project. In-memory is fine. If you want to persist data across server restarts, write the array to a JSON file using Node's fs module. That's your 'database' for now.

You have everything you need. Start Phase 1 today.